

FISICC

Ciencias de la computación 8

Protocolo de transmisión de imágenes

Martin Merida – 23002494

Daniel Pérez - 23002455

Contenido

Proyecto	3
Cómo ejecutar el proyecto	3
Servidor Java	3
Cliente React	3
Preprocesamiento de las imágenes	4
Transporte y sesión	4
Datos y coordenadas	5
Formato de los mensajes	6
1. Apertura y catálogo	6
2. Vista general ROOT	7
3. Vista y streams de tiles	7
4. Confirmación desalojo y cancelación	8
Planificación de tiles	8
Estado y coordinación de la entrega	9
Algoritmos de caché	10
1. Caché del cliente	10
Ejemplo del reloj circular	11
2. Caché global del servidor	11
Navegación y presentación	12
Errores cierre y límites	12
Logs del servidor	13
Resumen de mensajes del protocolo	14
Referencias	15

Proyecto

El sistema permite explorar imágenes grandes sin transferir al navegador toda su resolución original. Un conversor previo produce una pirámide de archivos RAW y un archivo de metadatos. El servidor lee las regiones necesarias y el cliente las dibuja en Canvas 2D.

Esta especificación cubre el protocolo de aplicación y las decisiones de navegación que afectan sus solicitudes. La preparación de imágenes es un proceso separado. La interfaz React se sirve por HTTP de forma independiente, el proceso Java atiende WebSocket en `ws://localhost:8080/ws`, o por medio de un túnel generado por ngrok en `wss://unhoarded-digna-snowless.ngrok-free.dev/ws`.

Cómo ejecutar el proyecto

Se requiere JDK 21 y Node.js 22.12 o superior con npm. El servidor Java y el cliente React se ejecutan en terminales separadas. Los comandos siguientes son para Linux o WSL, desde la carpeta raíz del proyecto.

El código Java debe estar en `Server/src` y el cliente, con `package.json`, en `Client`. Coloca cada carpeta de imagen convertida, con su `.meta` y sus RAW, dentro de `Server/Imagenes/images`. El proceso de conversión se describe en el apartado siguiente, el servidor consume esos archivos ya preparados.

Servidor Java

```
cd Server
mkdir -p build/classes
javac -d build/classes src/*.java
java -Xms128m -Xmx512m -cp build/classes Main
```

Main busca Imagenes/images desde el directorio actual y sus antecesores. La consola muestra la ruta encontrada y el puerto 8080. El endpoint local es `ws://localhost:8080/ws`. El presupuesto predeterminado de caché es 64 MiB y permite cuatro preparaciones simultáneas.

Cliente React

En otra terminal, desde la carpeta raíz del proyecto:

```
cd Client
npm ci
npm run dev
```

Abre la dirección que indique Vite, normalmente `http://localhost:5173`. En Detalles de conexión y carga, introduce `ws://localhost:8080/ws` y pulsa Reconectar para usar el servidor local. Si se

usa el túnel ngrok, la dirección WebSocket comienza con wss:// y termina en /ws, como la indicada al inicio del documento. El campo no acepta http:// ni https://.

Cuando los cuatro canales estén conectados, pulsa Cargar imágenes y después Ver imagen. ROOT utiliza RGBA4444 por defecto. Los botones de zoom, la rueda, la cruceta y el arrastre permiten explorar la imagen. Ctrl+C detiene cada proceso en su terminal.

Para generar y comprobar la versión de producción del cliente:

```
npm run build
npm run preview
```

La compilación queda en dist y la vista previa se abre normalmente en http://localhost:4173. El servidor Java continúa ejecutándose por separado. npm test ejecuta las pruebas del cliente.

Preprocesamiento de las imágenes

Primero que nada, para el preprocesamiento de las imágenes, las imágenes tienen que estar en formato .tif, para realizar esta conversión se usa un conversor externo al proyecto.

Una vez ya esta la imagen en .tif, se debe usar el conversor de imágenes para realizar la conversión. Desde la carpeta de Converter se ejecuta:

```
javac -encoding UTF-8 -cp ".;lib/*" ImageToRawConverter.java
java -cp ".;lib/*" ImageToRawConverter
```

y dentro del programa se selecciona la imagen que se desee convertir. La duración del proceso dependerá del tamaño de la imagen. Las imágenes grandes (93G+) podrían durar alrededor de 2 horas, sin embargo, mientras se esté realizando la conversión, se ira mostrando el porcentaje actual ya trabajado. Para la conversión de las imágenes, en los niveles donde la compresión es de $\frac{1}{2}$, $\frac{1}{4}$ y $\frac{1}{8}$ se va a realizar tomando un pixel de cada matriz de pixeles a convertir, mientras que para los niveles posteriores se va a calcular un promedio de cada matriz de pixeles, y para acelerar el proceso, cada nivel va a usar los promedios calculados por el nivel anterior. Las imágenes serán convertidas en una serie de archivos .raw (que contendrán los binarios de las imágenes en formato RGBA8888, en diferentes resoluciones hasta llegar a una resolución que se acople al nivel mínimo de 1024x1024, reduciendo cada nivel a la mitad de sus dimensiones). Además, se tendrá un archivo .meta que contiene la meta data de todos los archivos .raw generados. La carpeta de los archivos generados tendrá un peso menor o igual a 2 veces el peso de la imagen original.

Transporte y sesión

Cada cliente abre cuatro conexiones WebSocket mediante HTTP Upgrade, siguiendo el formato de transporte de RFC 6455. Cada conexión utiliza TCP. TCP ofrece entrega ordenada y retransmisión dentro de su conexión. Entre los cuatro canales no existe un orden común de llegada.

Canal	Función
CONTROL	Solicitudes, catálogo, ROOT, confirmaciones y avisos de desalojo.
CURRENT	Tiles centrales y vecinos del nivel actual.
PREVIOUS	Precarga del nivel anterior, cuando es mayor que cero.
NEXT	Precarga del nivel siguiente, si existe.

El servidor crea una sesión por navegador. Sus planes, confirmaciones y reservas son independientes de los de otros usuarios. Cada canal de datos tiene un worker que envía sus lotes secuencialmente, los tres workers pueden trabajar en paralelo. La implementación utiliza hilos virtuales de Java 21.

Datos y coordenadas

El nivel cero es ROOT y maxZoom es el nivel de mayor resolución. Las coordenadas parten de la esquina superior izquierda: X aumenta hacia la derecha y Y hacia abajo. Los tiles miden 256 × 256 píxeles.

Para cada nivel z , su lado virtual es $S(z) = \text{virtualSize} / 2^{(\text{maxZoom} - z)}$. Los niveles contiguos duplican su tamaño. El número de tiles por eje es $S(z) / 256$. La posición enviada en VIEWPORT es el origen de una ventana lógica de 4×4 tiles. En niveles mayores que cero, cada coordenada debe estar entre 0 y $S(z)/256 - 4$.

El archivo .meta define los datos siguientes. Las rutas de archivos son locales al servidor, el cliente solicita imágenes mediante su identificador.

Campo de metadatos	Significado
id, name	Identificador y nombre de la imagen.
originalWidth, originalHeight	Dimensiones de la imagen original.
virtualSize, tileSize	Lado virtual del nivel máximo y tamaño del tile.
level0Size, maxZoom	Tamaño de ROOT y último nivel disponible.
level.z.file	Archivo RAW correspondiente al nivel z .
level.z.virtualSize	Lado del cuadrado virtual de ese nivel.
level.z.width, level.z.height	Dimensiones de los píxeles reales guardados en el RAW.
level.z.offsetX, level.z.offsetY	Posición de esos píxeles dentro del cuadrado virtual.

Los RAW almacenan píxeles RGBA8888 por filas: R, G, B y A, un byte por componente. El servidor incorpora el espacio transparente indicado por los offsets al preparar ROOT y los tiles. El cliente NO DEBE sumar esos offsets nuevamente.

Los tiles viajan siempre en RGBA4444: dos bytes por píxel, con los bits RRRRGGGG BBBBAAAA, byte alto primero. Cada componente se obtiene tomando sus cuatro bits superiores, el cliente los multiplica por 17 al dibujar. Un tile ocupa 131072 bytes. ROOT permite RGBA4444 o RGBA8888, el valor predeterminado del cliente es RGBA4444. Los payloads contienen píxeles sin compresión JPEG o PNG.

Formato de los mensajes

Los comandos y cabeceras son mensajes de texto UTF-8. Los campos se separan con espacios y los enteros se escriben en decimal. Cada comando DEBE ocupar un mensaje WebSocket completo. Los signos < > de esta documentación delimitan campos variables y no se transmiten.

Campo	Uso y restricciones
sessionId	UUID creado por el servidor para unir los cuatro canales.
requestId	Identificador del cliente para ARCHIVOS y ROOT. Admite letras, dígitos, guion y guion bajo.
viewId	Identificador del cliente para una vista. Mismas restricciones que requestId. Cada nueva solicitud usa otro valor.
streamId	Identificador de lote generado por el servidor dentro de la sesión.
imageId	Identificador del catálogo. En ROOT y VIEWPORT admite también punto.
zoom, currentX, currentY	Nivel y origen entero de la ventana lógica. ROOT usa 0, 0, 0.
tileX, tileY	Coordenadas enteras del tile dentro de su nivel.
channel, format	Canal CURRENT, PREVIOUS o NEXT, formato RGBA4444 o RGBA8888 según la operación.

1. Apertura y catálogo

En el primer socket, que pasa a ser CONTROL:

```
SESSION_OPEN
SESSION_OK <sessionId>
```

En cada uno de los otros tres sockets, el cliente envía la primera línea y el servidor responde la segunda:

```
SESSION_JOIN <sessionId> <channel>
SESSION_JOINED <sessionId> <channel>
```

El cliente habilita las operaciones cuando los cuatro canales están listos. El catálogo se pide y se recibe por CONTROL:

```
ARCHIVOS <requestId>
ARCHIVOS_OK <requestId> <imageCount>
<imageId>|<name>|<width>|<height>|<virtualSize>|<maxZoom>
```

La última línea se repite una vez por imagen dentro del mismo mensaje. imageCount indica cuántas líneas contiene, width y height son las dimensiones originales. El cliente DEBE comprobar que la cantidad recibida coincida con la anunciada.

2. Vista general ROOT

Por CONTROL se intercambian los siguientes mensajes, en este orden:

```
ROOT <requestId> <imageId> <format>
ROOT_START <requestId> <imageId> <width> <height> <format> <chunkSize>
<chunkCount>
ROOT_DATA <requestId> <chunkIndex> <dataLength>
[mensaje binario del fragmento]
ROOT_END <requestId>
```

ROOT_START describe el cuadrado virtual completo. width y height son sus dimensiones, chunkSize es 65536 bytes y chunkCount es el total de fragmentos. La pareja ROOT_DATA y binario se repite, con chunkIndex desde cero. dataLength es la longitud exacta del binario siguiente, que puede ser menor en el último fragmento.

El cliente DEBE verificar formato, dimensiones, índices, longitudes y recepción completa antes de aceptar ROOT_END. No hay ACK por fragmento. ROOT queda en un canvas permanente mientras la imagen esté abierta y se libera el buffer de recepción.

Al abrir la imagen, el cliente envía también VIEWPORT <viewId> <imageId> 0 0 0, después de ROOT. CONTROL procesa primero la transmisión de ROOT y luego inicia la precarga del primer nivel por NEXT. Cuando maxZoom=0, no hay tiles que precargar. Volver a ROOT utiliza su copia local y vuelve a preparar NEXT sin descargar otra ROOT.

3. Vista y streams de tiles

El cliente solicita una vista por CONTROL:

```
VIEWPORT <viewId> <imageId> <zoom> <currentX> <currentY>
```

El servidor calcula el plan descrito en Planificación de tiles. Cada canal con tiles pendientes anuncia un lote mediante una sola cabecera de texto:

```
TILE_STREAM_START <streamId> <viewId> <imageId> <channel> <zoom> <format>
<tileSize> <tileBytes> <tileCount> <x1,y1> <x2,y2> ...
```

Campo del stream	Valor o significado
zoom	Nivel de todos los tiles de ese lote, siempre mayor que cero.
format, tileSize, tileBytes	RGBA4444, 256 y 131072, respectivamente.
tileCount	Cantidad de tiles anunciados. Puede ser menor que el plan por reutilización o reservas.
x1,y1 ...	Lista ordenada con exactamente tileCount pares de coordenadas.

Cada mensaje binario siguiente contiene un tile completo y corresponde a la siguiente coordenada de la lista. No incluye un identificador ni otra cabecera dentro del binario. El receptor DEBE comprobar su longitud y conservar ese orden. El lote termina con:

```
TILE_STREAM_END <streamId> <sentCount>
```

sentCount es la cantidad de tiles enviados, que DEBE coincidir con los binarios recibidos. Puede ser menor que tileCount al reemplazar o cancelar una vista: sólo se transmitió un prefijo de la lista. Terminar un stream no significa que toda la vista esté completa, otros canales o lotes pueden seguir pendientes.

4. Confirmación desalojo y cancelación

Estos mensajes se envían por CONTROL y no tienen una respuesta específica:

```
TILE_ACK <imageId> <zoom> <tileX> <tileY>
```

```
TILE_EVICT <imageId> <zoom> <tileX> <tileY>
```

```
CANCEL <viewId>
```

TILE_ACK confirma un tile válido recibido. Si se conserva en caché, queda registrado como disponible. Si se descarta, el cliente envía ACK seguido de EVICT. Una expulsión posterior también envía EVICT. Los duplicados que siguen almacenados reciben ACK y reutilizan el mismo buffer.

CANCEL detiene el plan sólo cuando su identificador coincide con la vista vigente. Un identificador antiguo no cancela una vista nueva. Se usa, por ejemplo, antes de abrir otra imagen. No cancela una transmisión ROOT que ya se está procesando en CONTROL.

Planificación de tiles

Para una vista con $z > 0$, el servidor construye estos grupos. Omite coordenadas que queden fuera de la imagen y no repite identidades.

Grupo	Región	Máximo	Canal
A	Centro de 4×4 en el nivel z .	16	CURRENT
B	Anillo exterior, incluidas las cuatro esquinas. Completa un cuadro de 6×6 .	20	CURRENT
C	Ventana de 4×4 en $z-1$, cuando $z > 1$.	16	PREVIOUS
D	Ventana de 4×4 en $z+1$, si existe.	16	NEXT

CURRENT envía primero A y después B. C y D se procesan en sus propios canales, sin esperar a CURRENT. Para cada eje, la posición del nivel siguiente es $2 \times (\text{current} + 1)$. La del anterior es $\text{floor}((\text{current} + 2) / 2) - 2$, limitada al rango válido de ese nivel.

El máximo del plan es 68 tiles. En una vista interior del último nivel hay 52 si existe PREVIOUS. Una vista ROOT no tiene grupos A, B ni C: solicita únicamente los 16 tiles centrales de $z=1$, desde $\text{floor}((S(1)/256 - 4)/2)$ en ambos ejes. Son los mismos que se usarán al hacer el primer zoom.

Estado y coordinación de la entrega

El servidor conserva estas estructuras por sesión:

Estructura	Contenido
clientHas	Conjunto de tiles que el cliente confirmó y no ha desalojado.
inFlight	Mapa de tiles reservados y stream propietario de cada reserva.
activePlans	Plan vigente de cada canal, asociado a la vista actual.
Generación del worker	Marca que permite detectar que un envío pertenece a un plan reemplazado.

Al recibir VIEWPORT, el servidor publica juntos los tres planes y despierta los workers. Cada worker reserva solamente tiles que no estén en clientHas ni en inFlight y cuyo plan siga vigente. Después los envía sin esperar un ACK entre tiles. Un TILE_ACK retira la reserva y añade la identidad a clientHas.

Al cambiar de vista, el worker puede terminar el tile en curso, cerrar parcialmente el lote y liberar las reservas que todavía le pertenecen. Una limpieza de un stream antiguo NO DEBE borrar reservas de otro. Cada liberación provoca una revisión de los faltantes de la vista vigente. Un EVICT retira el tile de clientes, si el plan todavía lo requiere, también provoca esa revisión.

Esto resuelve el caso en que NEXT reservó 16 tiles y sólo alcanzó a enviar diez antes de un zoom. CURRENT omite inicialmente los diez confirmados y los seis reservados. Al liberarse esas seis reservas, CURRENT los envía en otro lote, sin necesitar otra solicitud del usuario.

Un tile tardío se evalúa por (imageId, zoom, tileX, tileY), aunque venga de otra vista o canal. Así puede aprovecharse un tile enviado como NEXT que ahora pertenece a CURRENT. El registro de diagnóstico VIEW_COMPLETE indica que todos los tiles del plan están confirmados, no es un mensaje de red ni una confirmación de dibujo del canvas.

Estos mecanismos coordinan disponibilidad y cancelación. La aplicación no implementa stop-and-wait ni sliding window, y sus TILE_ACK no son ACK de TCP. No existen retransmisiones de aplicación por temporizador, el transporte y la revisión de planes cumplen funciones distintas.

Algoritmos de caché

1. Caché del cliente

Cada nivel activo tiene una lista circular Nth-chance Clock. CURRENT admite 36 tiles y los niveles adyacentes 16 cada uno. Los 16 centrales de CURRENT reciben $N=2$, los vecinos, incluidas las esquinas, y los tiles de otros niveles reciben $N=1$. ROOT permanece fuera de estas listas.

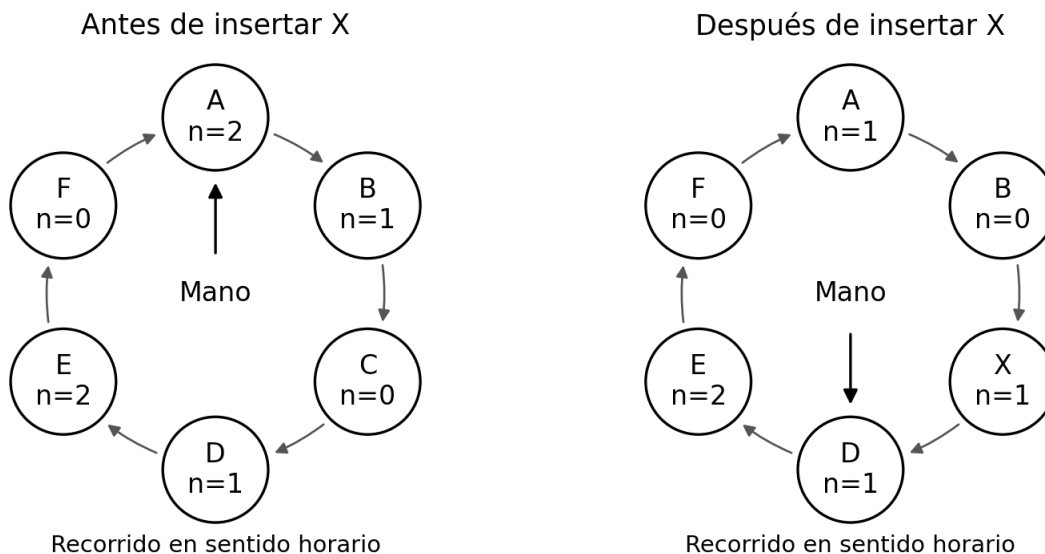
Para insertar un tile se utiliza primero un espacio libre. Si la lista está llena, la mano recorre sus entradas: decrementa los contadores positivos y avanza, al encontrar un cero reemplazable, expulsa esa entrada e inserta la nueva. Usar un tile al dibujarlo o reutilizarlo en el nuevo plan recarga su contador. El decremento ocurre durante la búsqueda de espacio, no por tiempo.

Los tiles del plan vigente están protegidos contra el reemplazo. Los que dejan de ser necesarios pueden permanecer hasta que haya presión de memoria. Al hacer zoom, las listas cambian de papel conservando los buffers útiles, primero se reducen cupos y después se amplían. Una expulsión libera también la superficie decodificada y envía TILE_EVICT antes del siguiente VIEWPORT cuando corresponde a esa rotación.

El límite es de 68 tiles, equivalentes a 8,5 MiB de payloads RGBA4444. No representa toda la memoria del navegador: existen también ROOT, el visor, una captura de transición y hasta 68 superficies decodificadas. En ROOT sólo se mantiene la lista NEXT de 16 tiles, si no hay otro nivel, no se crea ninguna lista.

Ejemplo del reloj circular

Los diagramas usan seis posiciones para mostrar el recorrido. En este ejemplo las entradas son reemplazables porque ya no pertenecen al plan vigente. Las listas reales mantienen los cupos descritos anteriormente.



Figuras 1 y 2. A comienza con $n=2$ y baja a 1, B pasa de 1 a 0, C ya tenía $n=0$ y se sustituye por X, que recibe $n=1$. La mano queda en D para la siguiente búsqueda. Si una entrada sigue protegida por el plan actual, tener $n=0$ no permite expulsarla. Un acceso vuelve a cargar su contador.

2. Caché global del servidor

La caché global comparte payloads preparados entre todas las sesiones: tiles y fragmentos de ROOT. Usa un reloj $N=2$ limitado por bytes, con 64 MiB de retención por defecto. No reserva esa cantidad por usuario ni guarda la imagen completa.

Su clave distingue imagen, versión del RAW, nivel, geometría, formato y región o fragmento. Un acierto reutiliza el payload y recarga sus oportunidades. Si varios usuarios piden simultáneamente el mismo contenido, comparten una preparación, se permiten hasta cuatro preparaciones distintas en paralelo. Un EVICT del navegador sólo modifica el estado de esa sesión y no vacía la caché global.

`image.cache.mib` configura entre 0 y 1024 MiB, cero desactiva la retención. `image.cache.loads` configura de 1 a 32 preparaciones simultáneas. El presupuesto cubre payloads retenidos, quedan fuera los estados de sesión, buffers temporales, envíos que conservan una referencia y demás memoria de la JVM.

Navegación y presentación

La cámara utiliza coordenadas virtuales del nivel cero, comunes a todos los niveles. Un tile de nivel z se ubica en $(\text{tileX} \times 256 / 2^z, \text{tileY} \times 256 / 2^z)$ y tiene lado $256 / 2^z$. El visor proyecta todo con la misma escala, por lo que ROOT y los tiles conservados siguen alineados.

El arrastre mueve continuamente la cámara. Para elegir la celda lógica se toma su centro: se resta la mitad de la ventana base de 4×4 , se divide entre el tamaño virtual de un tile y se redondea con floor(valor + 0,5). El resultado se limita a las posiciones válidas. Sólo se envía VIEWPORT al cambiar esa celda. El anillo de vecinos cubre los 25 tiles que puede intersectar una vista fraccional normal.

La cruceta lleva la vista a una posición alineada mediante una transición. El movimiento tiene un límite vectorial de 512 píxeles del canvas por segundo, también en diagonal. El zoom entre niveles dura aproximadamente un segundo y bloquea otras acciones de navegación durante la transición. ROOT permanece al fondo y una captura de la vista anterior cubre la llegada del detalle. ROOT se conserva en memoria, cuando el detalle cubre toda la cámara y terminó su fundido, el cliente omite repintar las capas inferiores que quedarían ocultas. Los tiles precargados no reinician su fundido al cambiar de vista.

En el último nivel, el cliente permite aumentos visuales de 2x, 4x y 8x. Se amplían los píxeles disponibles sin solicitar otro nivel. La cámara muestra una región menor y el mismo límite de velocidad hace que se recorra menos imagen. Cambiar de celda todavía envía VIEWPORT con el nivel máximo. Al reducir el aumento, se vuelve primero a 1x y después se cambia al nivel anterior. Una imagen que sólo tiene ROOT también permite esta ampliación local.

El área visible del canvas es cuadrada, además de su resolución interna de 1024×1024 . El espacio transparente que forme parte de la imagen preparada se conserva.

Errores cierre y límites

El servidor informa errores de aplicación con `ERROR <referenceId> <code> <message>`. La referencia identifica la solicitud o vista relacionada, vale cero cuando no aplica. El mensaje final puede contener espacios.

Código	Causa
SESSION_REQUIRED, SESSION_NOT_FOUND	Inicio incorrecto o sesión inexistente.
BAD_CHANNEL, DATA_CHANNEL_READ_ONLY	Canal inválido o comando enviado por un canal de datos.
BAD_REQUEST, UNKNOWN_COMMAND	Sintaxis inválida o comando no reconocido.
FILE_LIST_ERROR, IMAGE_NOT_FOUND	Error de catálogo o imagen no disponible.
UNSUPPORTED_FORMAT, ROOT_READ_ERROR	Formato no admitido o fallo de lectura de ROOT.
INVALID_VIEWPORT	Nivel, coordenadas o jerarquía de imagen inválidos.

Si CONTROL se cierra, el servidor elimina la sesión y detiene sus workers. Si falla un canal de datos, el cliente detiene la navegación y permite reconectar. No se reanuda automáticamente una sesión anterior. Un fallo de lectura durante un stream puede impedir que llegue su END, no debe interpretarse como un lote completo.

La implementación recibe mensajes WebSocket sin fragmentación, exige máscara al cliente y limita sus payloads a 16 MiB. Para ROOT, una fila debe caber en 65536 bytes y dividir exactamente ese tamaño. La aplicación no añade checksum propio, autenticación ni cifrado TLS, el endpoint directo utiliza ws://. Estos límites describen la versión implementada.

Logs del servidor

Los registros se imprimen en la consola: los eventos normales van a la salida estándar y los fallos a la salida de error. Como varios usuarios y canales trabajan en paralelo, sus líneas pueden intercalarse. Para seguir una operación se relacionan session, view, stream y la identidad del tile.

[WS] y [SESSION] muestran aperturas, cierres y unión de canales. [CONTROL <sessionId>] registra el comando recibido. [ROOT] indica el inicio y fin de la vista general, [VIEWPORT] muestra la posición y cuántos tiles se planificaron para CURRENT, PREVIOUS y NEXT.

[STREAM CURRENT], [STREAM PREVIOUS] y [STREAM NEXT] registran los lotes y su final, con enviados=<cantidad>/<anunciados>. [TILE_ACK] confirma recepción y [TILE_EVICT] registra un desalojo. [CANCEL] indica que se canceló la vista vigente. Un final parcial puede corresponder a una vista reemplazada, no demuestra por sí solo pérdida de datos.

Ejemplo con identificadores abreviados:

```
[STREAM CURRENT] session=S stream=17 view=12 z=8 tiles=16
[STREAM CURRENT] fin stream=17 enviados=16/16
[VIEW_COMPLETE] session=S view=12 confirmed=52/52
```

16/16 se refiere a ese lote, 52/52, al plan completo de esa vista. [VIEW_COMPLETE] aparece cuando todos sus tiles están confirmados por el cliente. Es un log del servidor, no un mensaje del protocolo ni una confirmación de que el canvas ya haya terminado de dibujar.

[CACHE GLOBAL], al iniciar el servidor y cerrar una sesión, muestra maxBytes y residentBytes (presupuesto y bytes retenidos), entries (payloads guardados), loading (preparaciones en curso), hits (aciertos), loads (preparaciones realizadas), sharedLoads (preparaciones compartidas), failures (fallos) y evictions (reemplazos). Las estadísticas corresponden a la caché compartida por todos los usuarios.

No se crea un archivo de logs automáticamente. Para guardar ambas salidas desde Server, se puede iniciar el proceso así:

```
java -Xmx512m -cp build/classes Main > servidor.log 2>&1
```

Resumen de mensajes del protocolo

C es el cliente y S el servidor. Datos corresponde a CURRENT, PREVIOUS o NEXT. Los formatos y campos se detallan en Formato de los mensajes, esta tabla reúne las operaciones de aplicación y sus payloads.

Mensaje	Sentido	Canal	Función
SESSION_OPEN	C → S	CONTROL	Abre una sesión en el primer socket.
SESSION_OK	S → C	CONTROL	Devuelve el sessionId de la sesión creada.
SESSION_JOIN	C → S	Datos	Une el socket a la sesión y al canal indicado.
SESSION_JOINED	S → C	Datos	Confirma la unión de ese canal.
ARCHIVOS	C → S	CONTROL	Solicita el catálogo de imágenes.
ARCHIVOS_OK	S → C	CONTROL	Envía cantidad e imágenes del catálogo en el mismo texto.
ROOT	C → S	CONTROL	Solicita la vista general y su formato de píxeles.
ROOT_START	S → C	CONTROL	Anuncia dimensiones, formato, tamaño y cantidad de fragmentos.
ROOT_DATA	S → C	CONTROL	Anuncia el índice y la longitud del binario siguiente.
Binario ROOT	S → C	CONTROL	Contiene el fragmento anunciado por ROOT_DATA, sin otra cabecera.
ROOT_END	S → C	CONTROL	Finaliza la transmisión de ROOT para esa solicitud.

Mensaje	Sentido	Canal	Función
VIEWPORT	C → S	CONTROL	Define imagen, nivel y posición. En ROOT precarga el primer nivel.
TILE_STREAM_START	S → C	Datos	Anuncia el lote y la lista ordenada de coordenadas.
Binario de tile	S → C	Datos	Contiene un tile RGBA4444 de 128 KiB, en el orden anunciado.
TILE_STREAM_END	S → C	Datos	Cierra el lote e indica cuántos tiles se enviaron.
TILE_ACK	C → S	CONTROL	Confirma la recepción de un tile identificado por sus coordenadas.
TILE_EVICT	C → S	CONTROL	Informa que el cliente ya no conserva ese tile.
CANCEL	C → S	CONTROL	Cancela el plan si viewId coincide con la vista vigente.
ERROR	S → C	Afectado	Informa referencia, código y descripción de un error.

Los binarios de ROOT y de tiles son payloads sin nombre de comando. Ping, Pong y Close pertenecen al transporte WebSocket definido por RFC 6455, no son comandos nuevos de esta aplicación.

Referencias

- [1] W. Eddy, editor. RFC 9293, Transmission Control Protocol (TCP). Agosto de 2022. Referencia para la organización de la especificación y las responsabilidades de TCP. <https://www.rfc-editor.org/rfc/rfc9293.html>
- [2] I. Fette y A. Melnikov. RFC 6455, The WebSocket Protocol. Diciembre de 2011. Referencia del transporte WebSocket. <https://www.rfc-editor.org/rfc/rfc6455.html>